

Hundred Days of Coding in C Programing

Day-16

Name – Arayana Khare

Batch - 15

Sap id – 590042432

Write a menu-driven C program using switch-case to perform digitbased operations on a positive integer. Define one separate userdefined function for each menu option, with void return type and with a simple function call (without call by value/reference). Each function should read the required input, process it, and display the result.

Prepare the algorithm and test case table. Then write, compile, execute, and test the C program against the prepared use cases. Question 1 check whether the digits of a positive integer are in increasing order from left to right. Step – 1

Algorithm

1. Start.
2. Read a positive integer n.
3. Extract the last digit and store it in previous.
4. Remove the last digit from the number.
5. Extract the next digit and store it in digit.
6. Compare digit with previous.
7. If digit is greater than or equal to previous, the digits are not in strictly increasing order.
8. Otherwise, update previous = digit.
9. Repeat the process until all digits are checked.
10. If no violation is found, display that the digits are in increasing order from left to right.
11. Stop.

Step – 2 Test Case Table

Test Case	Input	Expected Output
TC01	12345	Digits are in increasing order
TC02	13579	Digits are in increasing order
TC03	12431	Digits are not in increasing order
TC04	54321	Digits are not in increasing order

Step – 3 Write code in C Programing

```
#include <stdio.h>
```

```
void increasingOrder()
```

```
{
```

```
int n, digit, previous, flag = 1;
```

```
printf("Enter a positive integer: ");
```

```
scanf("%d", &n);
```

```
previous = n % 10; n
```

```
= n / 10;
```

```
while (n > 0)
```

```
{
```

```
digit = n % 10;
```

```
if (digit >= previous)
```

```
{
```

```
flag = 0; break;
```

```
}
```

```
previous = digit;  
n = n / 10;  
}
```

```
if (flag == 1)  
printf("Digits are in increasing order from left to right.\n"); else  
printf("Digits are not in increasing order from left to right.\n");  
}
```

```
int main()  
{  
int choice;
```

```
printf("===== DIGIT ORDER CHECK =====\n"); printf("1.  
Check whether digits are in increasing order\n"); printf("2.  
Exit\n");
```

```
printf("Enter your choice: "); scanf("%d",  
&choice);  
switch (choice)  
{  
case 1: increasingOrder(); break; case 2:  
printf("Program terminated successfully.\n");  
break; default:  
printf("Invalid choice!\n");  
}  
return 0;  
}
```

Step – 4 Compilation and Execution Case: 1

```
29     else
30     |     printf("Digits are not in increasing order from left to right.\n");
31     }
32
33 int main()
34 {
35     int choice;
36
37     printf("==== DIGIT ORDER CHECK ==== \n");
38     printf("1. Check whether digits are in increasing order\n");
39     printf("2. Exit\n");
40
41     printf("Enter your choice: ");
42     scanf("%d", &choice);
43
44     switch (choice)
45     {
46     |     case 1:
47     |         increasingOrder();
48     |         break;
49     |
50     |     case 2:
51     |         printf("Program terminated successfully.\n");
52     |         break;
53     |
54     |     default:
55     |         printf("Invalid choice!\n");
56     |     }
57
58     return 0;
59 }
```

input

```
==== DIGIT ORDER CHECK ====
1. Check whether digits are in increasing order
2. Exit
Enter your choice: 1
Enter a positive integer: 12345
Digits are in increasing order from left to right.
```


Case: 2

```
29     else
30         printf("Digits are not in increasing order from left to right.\n");
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK ==== \n");
38         printf("1. Check whether digits are in increasing order\n");
39         printf("2. Exit\n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 increasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully.\n");
52                 break;
53
54             default:
55                 printf("Invalid choice!\n");
56         }
57
58         return 0;
59     }
```

input

```
==== DIGIT ORDER CHECK ====
1. Check whether digits are in increasing order
2. Exit
Enter your choice: 1
Enter a positive integer: 13579
Digits are in increasing order from left to right.
```

Case: 3

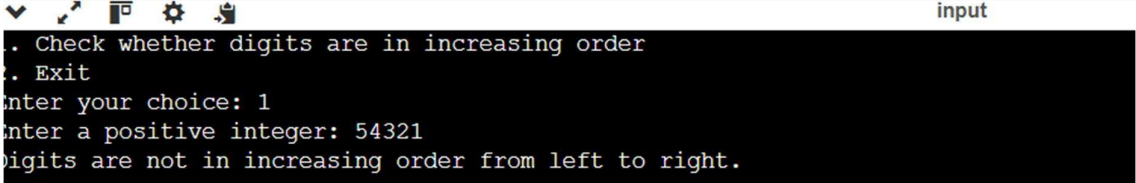
```
29     else
30         printf("Digits are not in increasing order from left to right.\n");
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK ==== \n");
38         printf("1. Check whether digits are in increasing order\n");
39         printf("2. Exit\n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 increasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully.\n");
52                 break;
53
54             default:
55                 printf("Invalid choice!\n");
56         }
57
58         return 0;
59     }
```

input

```
==== DIGIT ORDER CHECK ====
1. Check whether digits are in increasing order
2. Exit
Enter your choice: 1
Enter a positive integer: 12431
Digits are not in increasing order from left to right.
```

Case: 4

```
29     else
30         printf("Digits are not in increasing order from left to right.\n");
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK ==== \n");
38         printf("1. Check whether digits are in increasing order\n");
39         printf("2. Exit\n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 increasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully.\n");
52                 break;
53
54             default:
55                 printf("Invalid choice!\n");
56         }
57
58         return 0;
59     }
```



Step – 5 Conclusion

The program successfully checks whether the digits of a positive integer are arranged in strictly increasing order from left to right. It uses a switch-case statement and a separate void user-defined function to perform the required operation.

Question 2 check whether the digits of a positive integer are in decreasing order from left to right. Step – 1 Algorithm

1. Start.
2. Read a positive integer n.
3. Extract the last digit and store it in previous.
4. Remove the last digit from the number.
5. Extract the next digit and store it in digit.
6. Compare digit with previous.
7. If digit is less than or equal to previous, the digits are not in strictly decreasing order.
8. Otherwise, update previous = digit.
9. Repeat the process until all digits are checked.
10. If no violation is found, display that the digits are in decreasing order from left to right.
11. Stop.

Step – 2 Test Case Table

<u>Test Case</u>	<u>Input</u>	<u>Expected Output</u>
------------------	--------------	------------------------

<u>TC01</u>	<u>54321</u>	<u>Digits are in decreasing order</u>
<u>TC02</u>	<u>97531</u>	<u>Digits are in decreasing order</u>
<u>TC03</u>	<u>54325</u>	<u>Digits are not in decreasing order</u>
<u>TC04</u>	<u>12345</u>	<u>Digits are not in decreasing order</u>

Step – 3 Write Code in C Programing

```
#include <stdio.h>
```

```
void decreasingOrder()
```

```
{
```

```
int n, digit, previous, flag = 1;
```

```
printf("Enter a positive integer: ");
```

```
scanf("%d", &n);
```

```
previous = n % 10;
```

```
n = n / 10;
```

```
while (n > 0)
```

```
{
```

```
digit = n % 10;
```

```
if (digit <= previous)
```

```
{
```

```
flag = 0; break;
```

```
}
```

```
previous = digit;
```

```
n = n / 10;  
}
```

```
if (flag == 1)  
printf("Digits are in decreasing order from left to right.\n"); else  
printf("Digits are not in decreasing order from left to right.\n");  
}
```

```
int main()  
{  
int choice;
```

```
printf("==== DIGIT ORDER CHECK =====\n"); printf("1.  
Check whether digits are in decreasing order\n"); printf("2.  
Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch (choice)  
{  
case 1: decreasingOrder();  
break;
```

```
case 2:  
printf("Program terminated successfully.\n"); break;
```

```
default: printf("Invalid  
choice!\n");  
}
```

```
return 0;  
}
```

Step – 5 Compilation and Execution

Case: 1

```
29     else
30         printf("Digits are not in decreasing order from left to right.\n");
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK ==== \n");
38         printf("1. Check whether digits are in decreasing order\n");
39         printf("2. Exit\n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 decreasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully.\n");
52                 break;
53
54             default:
55                 printf("Invalid choice!\n");
56         }
57
58         return 0;
59     }
```

input

```
1. Check whether digits are in decreasing order
2. Exit
Enter your choice: 1
Enter a positive integer: 54321
Digits are in decreasing order from left to right.
```

Case: 2

```
29     else
30         printf("Digits are not in decreasing order from left
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK ==== \n");
38         printf("1. Check whether digits are in decreasing order \n");
39         printf("2. Exit \n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 decreasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully. \n");
52                 break;
53
54             default:
55                 printf("Invalid choice! \n");
56         }
57
58         return 0;
59     }
```

```
1. Check whether digits are in decreasing order
2. Exit
Enter your choice: 1
Enter a positive integer: 97531
Digits are in decreasing order from left to right.
```

Case: 3

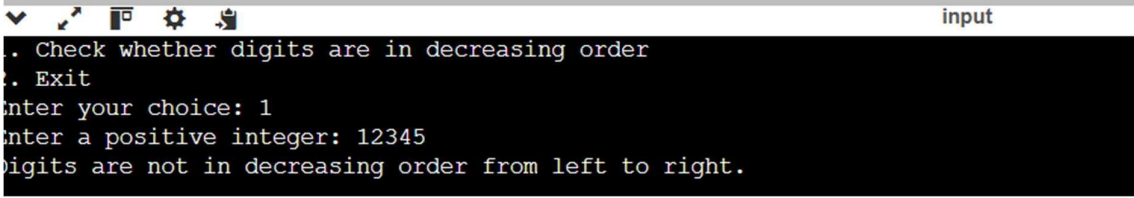
```
29     else
30         printf("Digits are not in decreasing order from left to right.\n");
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK ==== \n");
38         printf("1. Check whether digits are in decreasing order\n");
39         printf("2. Exit\n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 decreasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully.\n");
52                 break;
53
54             default:
55                 printf("Invalid choice!\n");
56         }
57
58         return 0;
59     }
```

input

```
1. Check whether digits are in decreasing order
2. Exit
Enter your choice: 1
Enter a positive integer: 54325
Digits are not in decreasing order from left to right.
```

Case: 4

```
29     else
30         printf("Digits are not in decreasing order from left to right.\n");
31     }
32
33     int main()
34     {
35         int choice;
36
37         printf("==== DIGIT ORDER CHECK =====\n");
38         printf("1. Check whether digits are in decreasing order\n");
39         printf("2. Exit\n");
40
41         printf("Enter your choice: ");
42         scanf("%d", &choice);
43
44         switch (choice)
45         {
46             case 1:
47                 decreasingOrder();
48                 break;
49
50             case 2:
51                 printf("Program terminated successfully.\n");
52                 break;
53
54             default:
55                 printf("Invalid choice!\n");
56         }
57
58         return 0;
59     }
```



. Check whether digits are in decreasing order
. Exit
Enter your choice: 1
Enter a positive integer: 12345
Digits are not in decreasing order from left to right.

Step – 5 Conclusion

The program successfully checks whether the digits of a positive integer are arranged in strictly decreasing order from left to right. It uses a switch-case statement and a separate void user-defined function without arguments to perform the required operation.

Question 3 find the largest and smallest digit present in a positive integer.

Step – 1 Algorithm

1. Start.
2. Read a positive integer n.

3. Initialize largest = 0 and smallest = 9.
4. Extract the last digit using $n \% 10$.
5. Compare the digit with largest and update largest if required.
6. Compare the digit with smallest and update smallest if required.
7. Remove the last digit using $n = n / 10$.
8. Repeat the process until all digits are checked.
9. Display the largest and smallest digits.
10. Stop.

Step – 2 Test Case Table

<u>Test Case</u>	<u>Input</u>	<u>Largest Digit</u>	<u>Smallest Digit</u>
<u>TC01</u>	<u>58321</u>	<u>8</u>	<u>1</u>
<u>TC02</u>	<u>90746</u>	<u>9</u>	<u>0</u>
<u>TC03</u>	<u>12345</u>	<u>5</u>	<u>1</u>
<u>TC04</u>	<u>76432</u>	<u>7</u>	<u>2</u>

Step – 3 Write a Code in C Programing

```
#include <stdio.h>

void largestSmallest()
{
    int n, digit; int
    largest = 0;
    int smallest = 9;

    printf("Enter a positive integer: "); scanf("%d",
    &n);

    while (n > 0)
    {
```

```
digit = n % 10;
```

```
if (digit > largest)
```

```
largest = digit;
```

```
if (digit < smallest)
```

```
smallest = digit;
```

```
n = n / 10;
```

```
}
```

```
printf("Largest digit = %d\n", largest);
```

```
printf("Smallest digit = %d\n", smallest);
```

```
}
```

```
int main()
```

```
{
```

```
int choice;
```

```
printf("==== DIGIT OPERATIONS =====\n");
```

```
printf("1. Find largest and smallest digit\n"); printf("2.
```

```
Exit\n");
```

```
printf("Enter your choice: ");
```

```
scanf("%d", &choice);
```

```
switch (choice)
```

```
{
```

```
case 1: largestSmallest();
```

```
break;
```

```
case 2: printf("Program terminated
```

```
successfully.\n"); break;
```

```
default:
```

```
printf("Invalid choice!\n");
```

```
}
```

```
return 0;
```

```
}
```

Step – 4 Compilation and Execution Case: 1

```
25     printf("Largest digit = %d\n", largest);
26     printf("Smallest digit = %d\n", smallest);
27 }
28
29 int main()
30 {
31     int choice;
32
33     printf("==== DIGIT OPERATIONS =====\n");
34     printf("1. Find largest and smallest digit\n");
35     printf("2. Exit\n");
36
37     printf("Enter your choice: ");
38     scanf("%d", &choice);
39
40     switch (choice)
41     {
42         case 1:
43             largestSmallest();
44             break;
45
46         case 2:
47             printf("Program terminated successfully.\n");
48             break;
49
50         default:
51             printf("Invalid choice!\n");
52     }
53
54     return 0;
55 }
```

```
==== DIGIT OPERATIONS =====
1. Find largest and smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 58321
Largest digit = 8
Smallest digit = 1
```

Case: 2

```
25     printf("Largest digit = %d\n", largest);
26     printf("Smallest digit = %d\n", smallest);
27 }
28
29 int main()
30 {
31     int choice;
32
33     printf("==== DIGIT OPERATIONS =====\n");
34     printf("1. Find largest and smallest digit\n");
35     printf("2. Exit\n");
36
37     printf("Enter your choice: ");
38     scanf("%d", &choice);
39
40     switch (choice)
41     {
42         case 1:
43             largestSmallest();
44             break;
45
46         case 2:
47             printf("Program terminated successfully.\n");
48             break;
49
50         default:
51             printf("Invalid choice!\n");
52     }
53
54     return 0;
55 }
```

```
==== DIGIT OPERATIONS =====
1. Find largest and smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 90746
Largest digit = 9
Smallest digit = 0
```

Case: 3

```
25     printf("Largest digit = %d\n", largest);
26     printf("Smallest digit = %d\n", smallest);
27 }
28
29 int main()
30 {
31     int choice;
32
33     printf("==== DIGIT OPERATIONS ==== \n");
34     printf("1. Find largest and smallest digit\n");
35     printf("2. Exit\n");
36
37     printf("Enter your choice: ");
38     scanf("%d", &choice);
39
40     switch (choice)
41     {
42         case 1:
43             largestSmallest();
44             break;
45
46         case 2:
47             printf("Program terminated successfully.\n");
48             break;
49
50         default:
51             printf("Invalid choice!\n");
52     }
53
54     return 0;
55 }
```

```
==== DIGIT OPERATIONS ====
1. Find largest and smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 12345
Largest digit = 5
Smallest digit = 1
```

Case: 4

```
25     printf("Largest digit = %d\n", largest);
26     printf("Smallest digit = %d\n", smallest);
27 }
28
29 int main()
30 {
31     int choice;
32
33     printf("==== DIGIT OPERATIONS ==== \n");
34     printf("1. Find largest and smallest digit\n");
35     printf("2. Exit\n");
36
37     printf("Enter your choice: ");
38     scanf("%d", &choice);
39
40     switch (choice)
41     {
42     case 1:
43         largestSmallest();
44         break;
45
46     case 2:
47         printf("Program terminated successfully.\n");
48         break;
49
50     default:
51         printf("Invalid choice!\n");
52     }
53
54     return 0;
55 }
```

```
==== DIGIT OPERATIONS ====
1. Find largest and smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 76432
Largest digit = 7
Smallest digit = 2
```

Step – 5 Conclusion

The program successfully finds the largest and smallest digit of a positive integer using a switch-case and a separate void function.

Question 4 find the second largest and second smallest distinct digits present in a positive integer. Step – 1 Algorithm

1. Start.
2. Read a positive integer n.
3. Initialize largest = -1, secondLargest = -1, smallest = 10, and secondSmallest = 10.
4. Extract each digit from the number.
5. Compare the digit with largest and update largest and secondLargest.
6. Compare the digit with smallest and update smallest and secondSmallest.
7. Ignore repeated occurrences of the same digit when determining the second values.
8. Continue until all digits are checked.
9. Display the second largest and second smallest distinct digits.
10. If fewer than two distinct digits exist, display that the required value is not available.
11. Stop.

Step – 2 Test Case Table

<u>Test Case</u>	<u>Input</u>	<u>Second Largest</u>	<u>Second Smallest</u>
<u>TC01</u>	<u>58321</u>	<u>5</u>	<u>2</u>
<u>TC02</u>	<u>90746</u>	<u>7</u>	<u>4</u>
<u>TC03</u>	<u>12345</u>	<u>4</u>	<u>2</u>
<u>TC04</u>	<u>76432</u>	<u>6</u>	<u>3</u>

Step – 3 Write Code in C Programing

```
#include <stdio.h>
```

```
void secondLargestSmallest()
```

```
{
int n, digit;
int largest = -1, secondLargest = -1;
int smallest = 10, secondSmallest = 10;

printf("Enter a positive integer: ");
scanf("%d", &n);

while (n > 0)
{
digit = n % 10;

if (digit > largest)
{
secondLargest = largest;
largest = digit;
}
else if (digit > secondLargest && digit != largest)
{
secondLargest = digit;
}

if (digit < smallest)
{
secondSmallest = smallest;
smallest = digit;
}
else if (digit < secondSmallest && digit != smallest)
{
secondSmallest = digit;
}

n = n / 10;
}

if (secondLargest == -1)
```

```
printf("Second largest distinct digit is not available.\n"); else  
printf("Second largest distinct digit = %d\n", secondLargest);
```

```
if (secondSmallest == 10)  
printf("Second smallest distinct digit is not available.\n"); else  
printf("Second smallest distinct digit = %d\n", secondSmallest);  
}
```

```
int main()  
{  
int choice;
```

```
printf("===== DIGIT OPERATIONS =====\n"); printf("1.  
Find second largest and second smallest digit\n"); printf("2.  
Exit\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
switch (choice)  
{  
case 1: secondLargestSmallest();  
break;
```

```
case 2:  
printf("Program terminated successfully.\n");  
break;
```

```
default: printf("Invalid  
choice!\n");  
}
```

```
return 0;  
}
```

Step – 4 Compilation and Execution

Case: 1

```
46     else
47         printf("Second smallest distinct digit = %d\n", secondSmallest);
48     }
49
50     int main()
51     {
52         int choice;
53
54         printf("==== DIGIT OPERATIONS =====\n");
55         printf("1. Find second largest and second smallest digit\n");
56         printf("2. Exit\n");
57
58         printf("Enter your choice: ");
59         scanf("%d", &choice);
60
61         switch (choice)
62         {
63             case 1:
64                 secondLargestSmallest();
65                 break;
66
67             case 2:
68                 printf("Program terminated successfully.\n");
69                 break;
70
71             default:
72                 printf("Invalid choice!\n");
73         }
74
75         return 0;
76     }
```

input

```
==== DIGIT OPERATIONS =====
1. Find second largest and second smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 58321
Second largest distinct digit = 5
Second smallest distinct digit = 2
```


Case: 2

```
46     else
47         printf("Second smallest distinct digit = %d\n", secondSmallest);
48     }
49
50     int main()
51     {
52         int choice;
53
54         printf("==== DIGIT OPERATIONS =====\n");
55         printf("1. Find second largest and second smallest digit\n");
56         printf("2. Exit\n");
57
58         printf("Enter your choice: ");
59         scanf("%d", &choice);
60
61         switch (choice)
62         {
63             case 1:
64                 secondLargestSmallest();
65                 break;
66
67             case 2:
68                 printf("Program terminated successfully.\n");
69                 break;
70
71             default:
72                 printf("Invalid choice!\n");
73         }
74
75         return 0;
76     }
```

input

```
==== DIGIT OPERATIONS =====
1. Find second largest and second smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 90746
Second largest distinct digit = 7
Second smallest distinct digit = 4
```

Case: 3

```
46     else
47     |     printf("Second smallest distinct digit = %d\n", secondSmallest);
48 }
49
50 int main()
51 {
52     int choice;
53
54     printf("==== DIGIT OPERATIONS ==== \n");
55     printf("1. Find second largest and second smallest digit\n");
56     printf("2. Exit\n");
57
58     printf("Enter your choice: ");
59     scanf("%d", &choice);
60
61     switch (choice)
62     {
63     |     case 1:
64     |         secondLargestSmallest();
65     |         break;
66     |
67     |     case 2:
68     |         printf("Program terminated successfully.\n");
69     |         break;
70     |
71     |     default:
72     |         printf("Invalid choice!\n");
73     |
74     }
75     return 0;
76 }
```

input

```
==== DIGIT OPERATIONS ====
1. Find second largest and second smallest digit
2. Exit
Enter your choice: 1
Enter a positive integer: 12345
Second largest distinct digit = 4
Second smallest distinct digit = 2
```

Case: 4

```
46     else
47         printf("Second smallest distinct digit = %d\n", secondSmallest);
48     }
49
50     int main()
51     {
52         int choice;
53
54         printf("==== DIGIT OPERATIONS =====\n");
55         printf("1. Find second largest and second smallest digit\n");
56         printf("2. Exit\n");
57
58         printf("Enter your choice: ");
59         scanf("%d", &choice);
60
61         switch (choice)
62         {
63             case 1:
64                 secondLargestSmallest();
65                 break;
66
67             case 2:
68                 printf("Program terminated successfully.\n");
69                 break;
70
71             default:
72                 printf("Invalid choice!\n");
73         }
74
75         return 0;
76     }
```

input

```
2. Exit
Enter your choice: 1
Enter a positive integer: 76432
Second largest distinct digit = 6
Second smallest distinct digit = 3
```

Step – 5 Conclusion

The program successfully finds the second largest and second smallest distinct digits of a positive integer. Repeated digits are ignored while determining the second largest and second smallest values.